RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23334 – Fundamentals of Data Science Lab

Laboratory Record Note Book

Name : SANJANA. S

Year / Branch / Section : IInd year CSE - H

University Register No. : 211625070164 7

College Roll No. : 250701647

Semester : III

Academic Year : 2026 - 27

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 – Fundamentals of Data Science Lab

NAME	SANJANA . S
ROLL NO	2507016 47
YEAR	II
SEC	H

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: SANJANA.S

Academic Year: 2026-27 **Semester:** III **Branch:** CSE

Register No. 211625070164 7

Certified that this is the bonafide record of work done by the above student in the CS23334- FUNDAMENTALS OF DATA SCIENCE *Laboratory during the academic year 2026 – 2027.*

3/11/26

Signature of Faculty in-charge

Submitted for the Practical Examination held on 06/11/2026

Internal Examiner

External Examiner

INDEX

Sl.NO	Experiment	Date	GitHub
1.a	Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib /seaborn for visualization.	29/07/26	
1.b	Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.	29/07/26	
1.c	Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.	05/08/26	
1.d	Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.	05/08/26	
2	Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.	12/08/26	
3.a	Conduct an experiment to show data visualization using line plot	19/08/26	
3.b	Conduct an experiment to show data visualization using bar chart	19/08/26	
3.c	Conduct an experiment to show data visualization using pie chart	19/08/26	
4	To Count the frequency of occurrence of a word in a body of text is often needed during text processing.	26/08/26	
5	Data Collection and Initial Exploration	16/09/26	
6	Time Series Analysis of Website Traffic	16/09/26	

7	Random Sampling and Sampling Distribution	23/09/26	
8	Conduct Z test for the Data given using Python	23/09/26	
9	Experiment to understand Matplotlib library use cases in Data Science through visualization.	23/09/26	
10	Experiment to understand array function in Data science.	23/09/26	
11	Experiment to understand pandas library use cases in Data science.	30/09/26	
12	Experiment to detect outliers in a given data set.	30/09/26	
13	Experiment to handle missing values and inappropriate data in a given data set.	30/09/26	
14	Experiment to understand feature scaling	30/09/26	
15	Experiment to understand the data preprocessing in Data science	30/09/26	
16	Experiment to understand EDA-Quantitative and Qualitative analysis.	07/10/26	
17	Experiment to understand Linear Regression for a given data set.	07/10/26	
18	Experiment to understand KNN algorithm for a given data set.	07/10/26	
19	Experiment to understand Logistic Regression for a given data set.	07/10/26	
20	Experiment to understand K-means clustering algorithm for a given data set.	07/10/26	

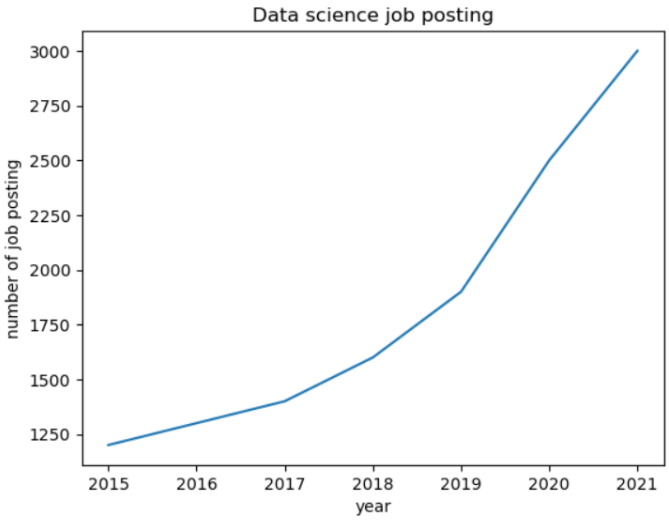
FUNDAMENTAL OF DATASCIENCE

1.a. Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib for visualization.

Program:

```
import matplotlib.pyplot as plt
x=[2015,2016,2017,2018,2019,2020,2021]
y=[1200,1300,1400,1600,1900,2500,3000]
plt.plot(x,y)
plt.title('Data science job posting')
plt.xlabel('year')
plt.ylabel('number of job posting')
plt.show()
```

Output:



Result:

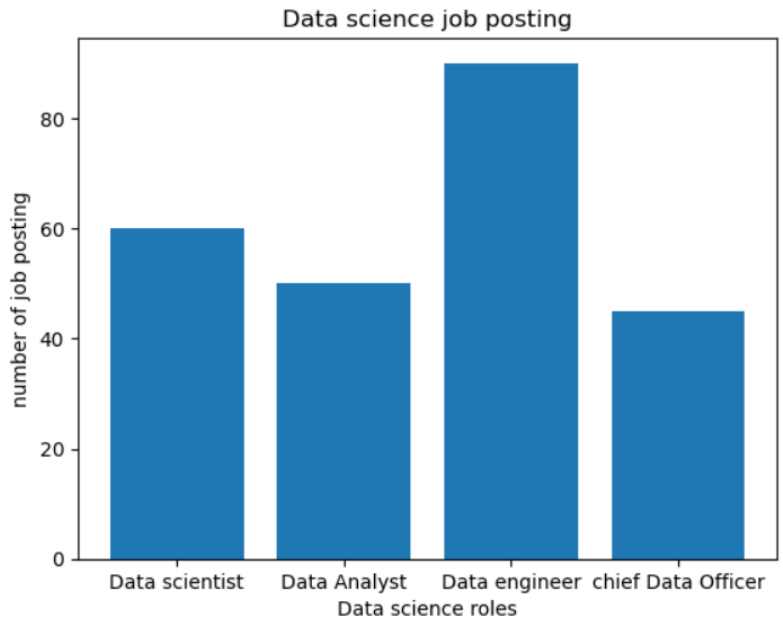
The trend of data science job posting over the decade was analyzed.

1.b.Analyze and visualize the distribution of various data science roles (Data Analyst,Data Engineer, Data Scientist, etc.) from a dataset.

Program:

```
import matplotlib.pyplot as plt
x=['Data scientist','Data Analyst','Data engineer','chief Data Officer']
y=[60,50,90,45]
plt.bar(x,y)
plt.title('Data science job posting')
plt.xlabel('Data science roles')
plt.ylabel('number of job posting')
plt.show()
```

Output:



Result:

The distribution of various data science roles was analysed.

1.c.Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data setsgiven.

Program:

```
import pandas as pd
structured_data = pd.DataFrame({
    'ID': [101, 102, 103],
    'Name': ['John', 'David', 'Emma'],
    'Age': [22, 28, 24]
})
print("Structured Data:\n", structured_data)
unstructured_data = "The weather today is sunny. It may include text, images, audio, or video."
print("\nUnstructured Data:\n", unstructured_data)
semi_structured_data = {
    'ID': 101,
    'Name': 'John',
    'Attributes': {
        'Height': 170,
        'Weight': 65
    }
}
print("\nSemi-structured Data:\n", semi_structured_data)
```

Output:

```
Structured Data:
     ID   Name  Age
0  101   John   22
1  102  David   28
2  103   Emma   24

Unstructured Data:
 The weather today is sunny. It may include text, images, audio, or video.

Semi-structured Data:
 {'ID': 101, 'Name': 'John', 'Attributes': {'Height': 170, 'Weight': 65}}
```

Result:

An experiment to differentiate Structured , Unstructured and semi-structured data was conducted.

1.d.Conduct an experiment to encrypt and decrypt given sensitive data using the cryptography library fernet.

Program:

```
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
token = f.encrypt(b"Software engineering")
print(token)
f.decrypt(token)
key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"Computer Science Engineering."
cipher_text = cipher_suite.encrypt(plain_text)
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
```

print("Decrypted Data:", decrypted_text)

Output:

```
b'gAAAAABqex_MoGb5m1umJvATBsWMDu9E6CDnefQWokwxoXMu02U_D8XQJvLyKdKgJvU-IpX4UK84NKXXLmkmEhOWvXmkFI5oTS9DbppxjuWqNycakcGDKjk='
Original Data: b'Computer Science Engineering.'
Encrypted Data: b'gAAAAABqex_M0Kxf5sY6cMuxlsocTS8j0s4javPNgHxru-Ar8CcWcjlHAxx_WjxIvKOyh0wmEWnWJcNFBAMX09M0KtGYzEeHRuQyc61EMwx9_eHpJ2IEQQw='
Decrypted Data: b'Computer Science Engineering.'
```

Result:

An Experiment to encrypt and decrypt the data was conducted.

2) Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df = pd.DataFrame({
    'Date': ['2025-02-01', '2025-02-02', '2025-02-03', '2025-02-04', '2025-02-05',
             '2025-02-06', '2025-02-07', '2025-02-08'],
    'Product': ['Monitor', 'Keyboard', 'Mouse', 'Monitor', 'Keyboard',
                'Mouse', 'Monitor', 'Keyboard'],
    'Sales': [28000, 12000, 8000, 35000, 15000, 9500, 42000, 18000],
    'Quantity': [4, 10, 12, 5, 12, 15, 6, 14],
    'Region': ['East', 'West', 'North', 'South', 'East',
               'North', 'West', 'South']
})
print(df.head())
print(df.isnull().sum())
df['Sales'] = df['Sales'].fillna(df['Sales'].mean())
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
print(df.describe())
product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
    'Quantity': 'sum'
}).reset_index()
print(product_summary)
plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
```

```
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
df['Date'] = pd.to_datetime(df['Date'])
sales_over_time = df.groupby('Date')['Sales'].sum().reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'], marker='o')
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.xticks(rotation=45)
plt.show()
pivot_table = df.pivot_table(
    values='Sales',
    index='Region',
    columns='Product',
    aggfunc='sum',
    fill_value=0
)
print(pivot_table)
correlation_matrix = df.corr(numeric_only=True)
print(correlation_matrix)
plt.figure(figsize=(8, 6))
plt.imshow(correlation_matrix, cmap='coolwarm')
plt.colorbar()
plt.xticks(range(len(correlation_matrix.columns)), correlation_matrix.columns)
plt.yticks(range(len(correlation_matrix.columns)), correlation_matrix.columns)
plt.title('Correlation Matrix')
```

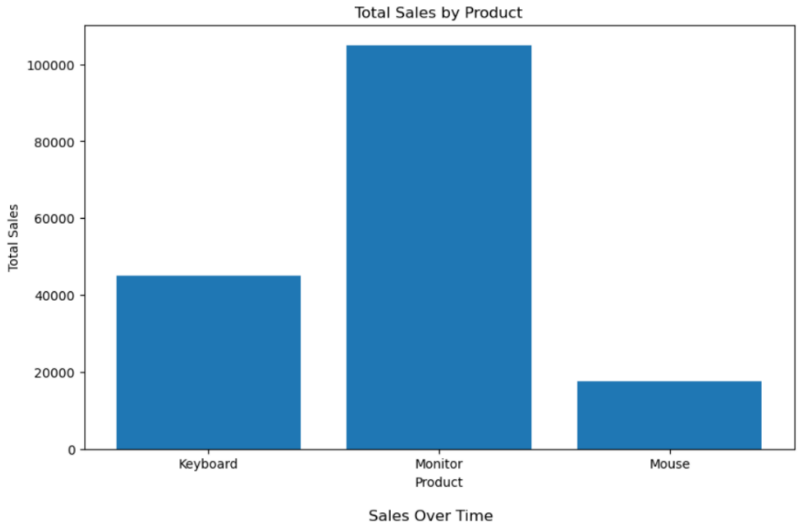
plt.show()

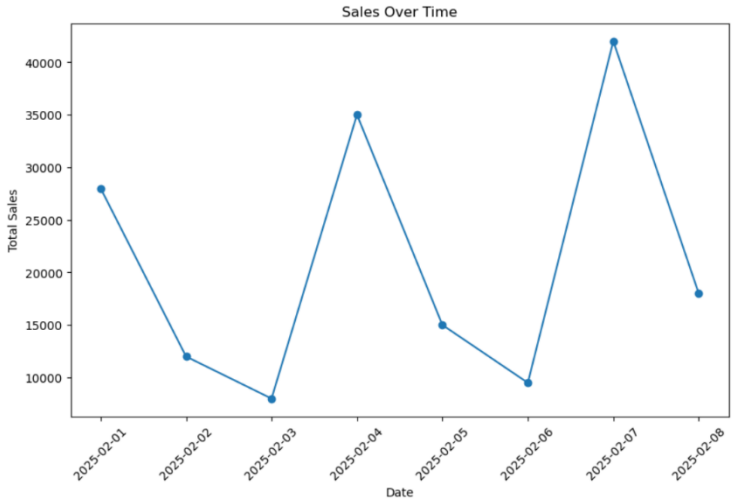
Output:

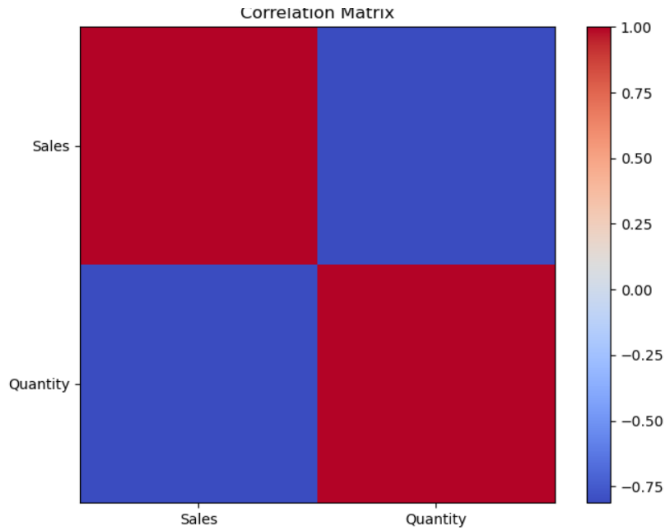
```
         Date   Product  Sales  Quantity Region
0  2025-02-01   Monitor  28000         4   East
1  2025-02-02  Keyboard  12000        10   West
2  2025-02-03     Mouse   8000        12  North
3  2025-02-04   Monitor  35000         5  South
4  2025-02-05  Keyboard  15000        12   East
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
              Sales   Quantity
count      8.000000   8.000000
mean   20937.500000   9.750000
std    12610.759295   4.234214
min     8000.000000   4.000000
25%    11375.000000   5.750000
50%    16500.000000  11.000000
75%    29750.000000  12.500000
max    42000.000000  15.000000
    Product   Sales  Quantity
0  Keyboard   45000        36
1   Monitor  105000        15
2     Mouse   17500        27
```

Product	Keyboard	Monitor	Mouse
Region			
East	15000	28000	0
North	0	0	17500
South	18000	35000	0
West	12000	42000	0

	Sales	Quantity
Sales	1.000000	-0.812317
Quantity	-0.812317	1.000000







Result:

The dataset in CSV format and performed data processing and visualised was analysed.

3) Handling Missing and Inappropriate Data in a Dataset

Program:

```
import numpy as np
import pandas as pd
df = pd.DataFrame({
    'CustomerID': [301, 302, 303, 304, 305, 306, 307, 308, 309, 309, 310],
    'Age_Group': ['18-23', '24-29', '30-35', '18-23', '36+', '36+',
                  '30-35', '18-23', '24-29', '24-29', '30-35'],
    'Rating(1-5)': [4, 2, 5, -2, 3, 7, 1, 4, 5, 5, 3],
    'Hotel': ['Taj', 'ITC', 'Marriott', 'ITC', 'Taj', 'Taj',
              'Marriott', 'ITC', 'Taj', 'Taj', 'Marriott'],
    'FoodPreference': ['Veg', 'Non-Veg', 'veg', 'Vegetarian', 'Veg',
                       'Non-Veg', 'Vegetarian', 'Veg', 'Non-Veg', 'Non-Veg', 'veg'],
    'Bill': [2200, 1800, 3200, -500, 1500, 2800, -900, 3500, 2400, 2400, 1900],
    'NoOfPax': [2, 4, 3, 5, 2, 22, -3, 6, 4, 4, 3],
    'EstimatedSalary': [52000, 68000, 75000, 85000, 62000, 71000,
                        -20000, 250000, 58000, 58000, 90000],
    'Age_Group.1': ['18-23', '24-29', '30-35', '18-23', '36+', '36+',
                    '30-35', '18-23', '24-29', '24-29', '30-35']
})
print("ORIGINAL DATASET")
print(df)
print("\nDUPLICATE ROWS")
print(df.duplicated())
df.drop_duplicates(inplace=True)
df.reset_index(drop=True, inplace=True)
df.drop(['Age_Group.1'], axis=1, inplace=True)
df.loc[
```

```
    (df['Rating(1-5)'] < 1) |
    (df['Rating(1-5)'] > 5),
    'Rating(1-5)'
] = np.nan
df.loc[df['Bill'] < 0, 'Bill'] = np.nan
df.loc[df['EstimatedSalary'] < 0, 'EstimatedSalary'] = np.nan
df.loc[
    (df['NoOfPax'] < 1) |
    (df['NoOfPax'] > 20),
    'NoOfPax'
] = np.nan
print("\nUNIQUE AGE GROUPS")
print(df['Age_Group'].unique())
print("\nUNIQUE HOTELS")
print(df['Hotel'].unique())
print("\nUNIQUE FOOD PREFERENCES")
print(df['FoodPreference'].unique())
df['FoodPreference'] = df['FoodPreference'].replace(
    ['veg', 'Vegetarian', 'Veg'],
    'Veg'
)
df['FoodPreference'] = df['FoodPreference'].replace(
    ['non-Veg', 'non-veg'],
    'Non-Veg'
)
df['EstimatedSalary'] = df['EstimatedSalary'].fillna(
    round(df['EstimatedSalary'].mean())
)
df['NoOfPax'] = df['NoOfPax'].fillna(
```

```
    round(df['NoOfPax'].median())
)

df['Rating(1-5)'] = df['Rating(1-5)'].fillna(
    round(df['Rating(1-5)'].median())
)
df['Bill'] = df['Bill'].fillna(
    round(df['Bill'].mean())
)
print("\nFINAL CLEANED DATASET")
print(df)
```

Output:

```
ORIGINAL DATASET
    CustomerID Age_Group  Rating(1-5)     Hotel FoodPreference  Bill  NoOfPax  \
0          301     18-23            4       Taj            Veg  2200        2
1          302     24-29            2       ITC        Non-Veg  1800        4
2          303     30-35            5  Marriott            veg  3200        3
3          304     18-23           -2       ITC     Vegetarian  -500        5
4          305       36+            3       Taj            Veg  1500        2
5          306       36+            7       Taj        Non-Veg  2800       22
6          307     30-35            1  Marriott     Vegetarian  -900       -3
7          308     18-23            4       ITC            Veg  3500        6
8          309     24-29            5       Taj        Non-Veg  2400        4
9          309     24-29            5       Taj        Non-Veg  2400        4
10         310     30-35            3  Marriott            veg  1900        3
```

```
    EstimatedSalary Age_Group.1
0             52000       18-23
1             68000       24-29
2             75000       30-35
3             85000       18-23
4             62000         36+
5             71000         36+
6            -20000       30-35
7            250000       18-23
8             58000       24-29
9             58000       24-29
10            90000       30-35

DUPLICATE ROWS
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool
```

```
UNIQUE AGE GROUPS
<StringArray>
['18-23', '24-29', '30-35', '36+']
Length: 4, dtype: str

UNIQUE HOTELS
<StringArray>
['Taj', 'ITC', 'Marriott']
Length: 3, dtype: str

UNIQUE FOOD PREFERENCES
<StringArray>
['Veg', 'Non-Veg', 'veg', 'Vegetarian']
Length: 4, dtype: str
```

```
FINAL CLEANED DATASET
   CustomerID Age_Group  Rating(1-5)     Hotel FoodPreference    Bill  \
0         301     18-23          4.0       Taj            Veg  2200.0
1         302     24-29          2.0       ITC        Non-Veg  1800.0
2         303     30-35          5.0  Marriott            Veg  3200.0
3         304     18-23          4.0       ITC            Veg  2412.0
4         305       36+          3.0       Taj            Veg  1500.0
5         306       36+          4.0       Taj        Non-Veg  2800.0
6         307     30-35          1.0  Marriott            Veg  2412.0
7         308     18-23          4.0       ITC            Veg  3500.0
8         309     24-29          5.0       Taj        Non-Veg  2400.0
9         310     30-35          3.0  Marriott            Veg  1900.0

   NoOfPax  EstimatedSalary
0      2.0          52000.0
1      4.0          68000.0
2      3.0          75000.0
3      5.0          85000.0
4      2.0          62000.0
5      4.0          71000.0
6      4.0          90111.0
7      6.0         250000.0
8      4.0          58000.0
9      3.0          90000.0
```

Result:

Inappropriate data in the dataset is successfully studied.

4) Experiment to understand the data preprocessing in Data science

Program:

```
import numpy as np
import pandas as pd
df = pd.DataFrame({
    'Country': ['USA', 'France', 'USA', 'Italy', 'France',
                'USA', 'Italy', np.nan, 'France', 'USA'],
    'Age': [27, 34, 29, np.nan, 43, 36, 31, 47, 28, 40],
    'Salary': [45000, 61000, 52000, 65000, np.nan, 55000,
               70000, 78000, 53000, 63000],
    'Purchased': ['Yes', 'No', 'No', 'Yes', 'No',
                  'Yes', 'Yes', 'No', 'Yes', 'No']
})
print("Original Dataset:")
print(df)
print("\n\nDataFrame Information:")
print(df.info())
print("\n\nCountry Mode:")
print(df.Country.mode())
print("\n\nFirst Mode Value:")
print(df.Country.mode()[0])
df['Country'] = df['Country'].fillna(df.Country.mode()[0])
df['Age'] = df['Age'].fillna(df.Age.median())
df['Salary'] = df['Salary'].fillna(round(df.Salary.mean()))
print("\n\nDataset After Filling Missing Values:")
print(df)
print("\n\nOne Hot Encoding:")
print(pd.get_dummies(df.Country))
```

```
updated_dataset = pd.concat(
    [pd.get_dummies(df.Country), df.iloc[:, [1, 2, 3]]],
    axis=1
)
print("\n\nUpdated Dataset:")
print(updated_dataset)
print("\n\nDataFrame Information After Preprocessing:")
print(df.info())
updated_dataset['Purchased'] = updated_dataset['Purchased'].replace(
    ['No', 'Yes'], [0, 1]
)
print("\n\nFinal Dataset:")
print(updated_dataset)
```

Ouput:

```
Original Dataset:
  Country   Age   Salary Purchased
0     USA  27.0  45000.0       Yes
1  France  34.0  61000.0        No
2     USA  29.0  52000.0        No
3   Italy   NaN  65000.0       Yes
4  France  43.0      NaN        No
5     USA  36.0  55000.0       Yes
6   Italy  31.0  70000.0       Yes
7     NaN  47.0  78000.0        No
8  France  28.0  53000.0       Yes
9     USA  40.0  63000.0        No
```

```
DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    9 non-null      str
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     str
dtypes: float64(2), str(2)
memory usage: 312.0 bytes
None
```

```
Country Mode:
0    USA
Name: Country, dtype: str

First Mode Value:
USA
```

Dataset After Filling Missing Values:

	Country	Age	Salary	Purchased
0	USA	27.0	45000.0	Yes
1	France	34.0	61000.0	No
2	USA	29.0	52000.0	No
3	Italy	34.0	65000.0	Yes
4	France	43.0	60222.0	No
5	USA	36.0	55000.0	Yes
6	Italy	31.0	70000.0	Yes
7	USA	47.0	78000.0	No
8	France	28.0	53000.0	Yes
9	USA	40.0	63000.0	No

One Hot Encoding:

	France	Italy	USA
0	False	False	True
1	True	False	False
2	False	False	True
3	False	True	False
4	True	False	False
5	False	False	True
6	False	True	False
7	False	False	True
8	True	False	False
9	False	False	True

Updated Dataset:

	France	Italy	USA	Age	Salary	Purchased
0	False	False	True	27.0	45000.0	Yes
1	True	False	False	34.0	61000.0	No
2	False	False	True	29.0	52000.0	No
3	False	True	False	34.0	65000.0	Yes
4	True	False	False	43.0	60222.0	No
5	False	False	True	36.0	55000.0	Yes
6	False	True	False	31.0	70000.0	Yes
7	False	False	True	47.0	78000.0	No
8	True	False	False	28.0	53000.0	Yes
9	False	False	True	40.0	63000.0	No

```
DataFrame Information After Preprocessing:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     str
 1   Age        10 non-null     float64
 2   Salary     10 non-null     float64
 3   Purchased  10 non-null     str
dtypes: float64(2), str(2)
memory usage: 312.0 bytes
None
```

Final Dataset:

	France	Italy	USA	Age	Salary	Purchased
0	False	False	True	27.0	45000.0	1
1	True	False	False	34.0	61000.0	0
2	False	False	True	29.0	52000.0	0
3	False	True	False	34.0	65000.0	1
4	True	False	False	43.0	60222.0	0
5	False	False	True	36.0	55000.0	1
6	False	True	False	31.0	70000.0	1
7	False	False	True	47.0	78000.0	0
8	True	False	False	28.0	53000.0	1
9	False	False	True	40.0	63000.0	0

Result:

The data preprocessing in the Data Science is understood.

5) Experiment to detect outliers in a given data set. Description: Understand the procedure to identify the outliers in a given dataset

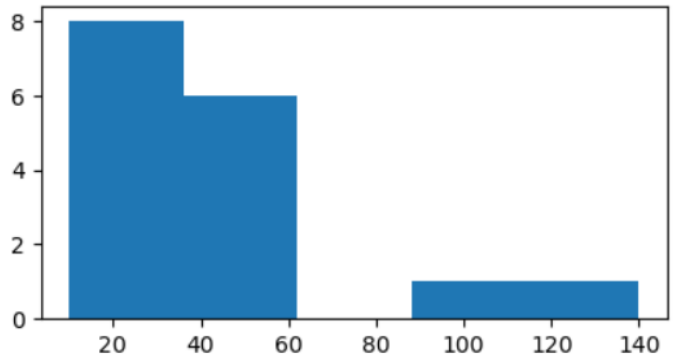
Program:

```
import numpy as np
import matplotlib.pyplot as plt
array=np.array([10,15,19,22,25,28,31,34,37,39,42,44,47,49,110,140])
print("Array:")
print(array)
print("Mean:")
print(array.mean())
print("25th Percentile:")
print(np.percentile(array,25))
print("50th Percentile:")
print(np.percentile(array,50))
print("75th Percentile:")
print(np.percentile(array,75))
print("100th Percentile:")
print(np.percentile(array,100))
def outDetection(array):
    Q1,Q3=np.percentile(array,[25,75])
    IQR=Q3-Q1
    lr=Q1-(1.5*IQR)
    ur=Q3+(1.5*IQR)
    return lr,ur
lr,ur=outDetection(array)
print("Lower Range:")
print(lr)
print("Upper Range:")
```

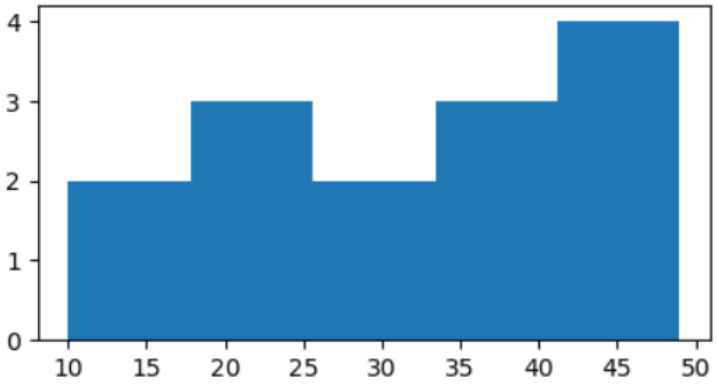
```
print(ur)
plt.figure(figsize=(5,2.5))
plt.hist(array,bins=5)
plt.show()
new_array=array[(array>lr)&(array<ur)]
print("Array After Removing Outliers:")
print(new_array)
plt.figure(figsize=(5,2.5))
plt.hist(new_array,bins=5)
plt.show()
lr1,ur1=outDetection(new_array)
print("New Lower Range:")
print(lr1)
print("New Upper Range:")
print(ur1)
final_array=new_array[(new_array>lr1)&(new_array<ur1)]
print("Final Array:")
print(final_array)
plt.figure(figsize=(5,2.5))
plt.hist(final_array,bins=5)
plt.show()
```

Output:

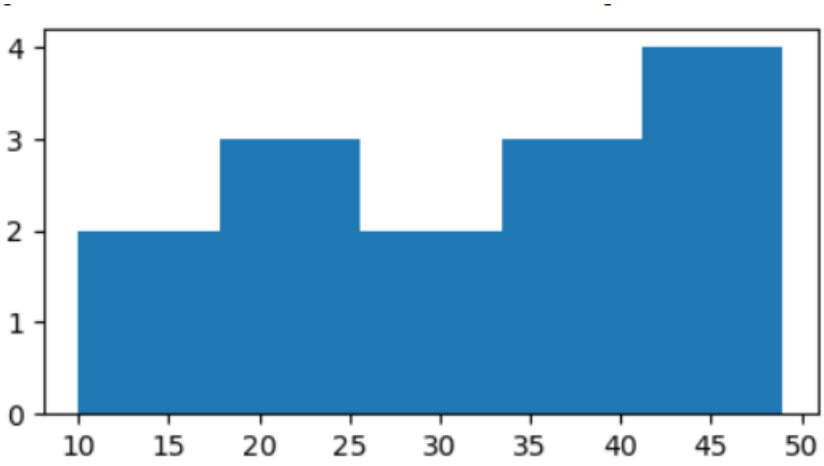
```
Array:
[ 10  15  19  22  25  28  31  34  37  39  42  44  47  49 110 140]
Mean:
43.25
25th Percentile:
24.25
50th Percentile:
35.5
75th Percentile:
44.75
100th Percentile:
140.0
Lower Range:
-6.5
Upper Range:
75.5
```




```
Array After Removing Outliers:
[10 15 19 22 25 28 31 34 37 39 42 44 47 49]
```




```
New Lower Range:
-5.0
New Upper Range:
69.0
Final Array:
[10 15 19 22 25 28 31 34 37 39 42 44 47 49]
```



Result:

Experiments to detect the outliers for the data is conducted.

6) Experiment to understand feature scaling. Description: Understand the importance of feature scaling

Program:

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
df=pd.DataFrame({

'Country':['USA','France','USA','China','France','USA','China',np.nan,'France','USA'],
    'Age':[22,34,27,np.nan,45,38,31,50,29,41],
    'Salary':[35000,60000,48000,72000,np.nan,55000,68000,80000,52000,64000],
    'Purchased':['No','Yes','No','Yes','No','Yes','No','Yes','No','Yes']
})
print("Original Dataset:")
print(df)
print("\n\nDataFrame Information:")
print(df.info())
df['Country']=df['Country'].fillna(df['Country'].mode()[0])
features=df.iloc[:,:-1].values
label=df.iloc[:,-1].values
age=SimpleImputer(strategy="mean",missing_values=np.nan)
Salary=SimpleImputer(strategy="mean",missing_values=np.nan)
age.fit(features[:,[1]])
Salary.fit(features[:,[2]])
features[:,[1]]=age.transform(features[:,[1]])
```

```
features[:,[2]]=Salary.transform(features[:,[2]])
print("\n\nAfter Filling Missing Values:")
print(features)
oh=OneHotEncoder(sparse_output=False)
Country=oh.fit_transform(features[:,[0]])
print("\n\nOne Hot Encoding:")
print(Country)
final_set=np.concatenate((Country,features[:,[1,2]]),axis=1)
print("\n\nFinal Set:")
print(final_set)
sc=StandardScaler()
sc.fit(final_set)
feat_standard_scaler=sc.transform(final_set)
print("\n\nStandard Scaler:")
print(feat_standard_scaler)
mms=MinMaxScaler(feature_range=(0,1))
mms.fit(final_set)
feat_minmax_scaler=mms.transform(final_set)
print("\n\nMin Max Scaler:")
print(feat_minmax_scaler)
```

Output:

```
Original Dataset:
  Country   Age   Salary Purchased
0     USA  22.0  35000.0        No
1  France  34.0  60000.0       Yes
2     USA  27.0  48000.0        No
3   China   NaN  72000.0       Yes
4  France  45.0      NaN        No
5     USA  38.0  55000.0       Yes
6   China  31.0  68000.0        No
7     NaN  50.0  80000.0       Yes
8  France  29.0  52000.0        No
9     USA  41.0  64000.0       Yes
```

```
DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    9 non-null      str
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     str
dtypes: float64(2), str(2)
memory usage: 312.0 bytes
None
```

```
After Filling Missing Values:
[['USA' 22.0 35000.0]
 ['France' 34.0 60000.0]
 ['USA' 27.0 48000.0]
 ['China' 35.22222222222222 72000.0]
 ['France' 45.0 59333.333333333336]
 ['USA' 38.0 55000.0]
 ['China' 31.0 68000.0]
 ['USA' 50.0 80000.0]
 ['France' 29.0 52000.0]
 ['USA' 41.0 64000.0]]
```

```
One Hot Encoding:
[[0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

```
Final Set:
[[0.0 0.0 1.0 22.0 35000.0]
 [0.0 1.0 0.0 34.0 60000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [1.0 0.0 0.0 35.22222222222222 72000.0]
 [0.0 1.0 0.0 45.0 59333.333333333336]
 [0.0 0.0 1.0 38.0 55000.0]
 [1.0 0.0 0.0 31.0 68000.0]
 [0.0 0.0 1.0 50.0 80000.0]
 [0.0 1.0 0.0 29.0 52000.0]
 [0.0 0.0 1.0 41.0 64000.0]]
```

```
Standard Scaler:
[[-5.00000000e-01 -6.54653671e-01  1.00000000e+00 -1.63305082e+00
  -2.00154053e+00]
 [-5.00000000e-01  1.52752523e+00 -1.00000000e+00 -1.50954278e-01
   5.48367268e-02]
 [-5.00000000e-01 -6.54653671e-01  1.00000000e+00 -1.01551060e+00
  -9.32224356e-01]
 [ 2.00000000e+00 -6.54653671e-01 -1.00000000e+00  0.00000000e+00
   1.04189781e+00]
 [-5.00000000e-01  1.52752523e+00 -1.00000000e+00  1.20763422e+00
   5.98484550e-16]
 [-5.00000000e-01 -6.54653671e-01  1.00000000e+00  3.43077904e-01
  -3.56438724e-01]
 [ 2.00000000e+00 -6.54653671e-01 -1.00000000e+00 -5.21478414e-01
   7.12877448e-01]
 [-5.00000000e-01 -6.54653671e-01  1.00000000e+00  1.82517445e+00
   1.69993853e+00]
 [-5.00000000e-01  1.52752523e+00 -1.00000000e+00 -7.68494505e-01
  -6.03203995e-01]
 [-5.00000000e-01 -6.54653671e-01  1.00000000e+00  7.13602040e-01
   3.83857088e-01]]
```

```
Min Max Scaler:
[[0.         0.         1.         0.         0.        ]
 [0.         1.         0.         0.42857143 0.55555556]
 [0.         0.         1.         0.17857143 0.28888889]
 [1.         0.         0.         0.47222222 0.82222222]
 [0.         1.         0.         0.82142857 0.54074074]
 [0.         0.         1.         0.57142857 0.44444444]
 [1.         0.         0.         0.32142857 0.73333333]
 [0.         0.         1.         1.         1.        ]
 [0.         1.         0.         0.25       0.37777778]
 [0.         0.         1.         0.67857143 0.64444444]]
```

Result:

Experiments to understand the Feature Scaling is conducted.

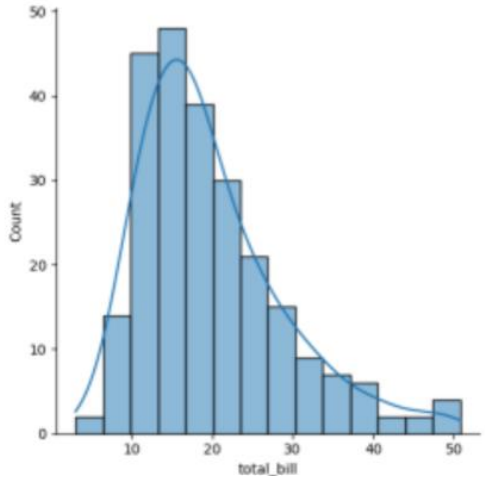
7) Experiment to understand EDA-Quantitative and Qualitative analysis.

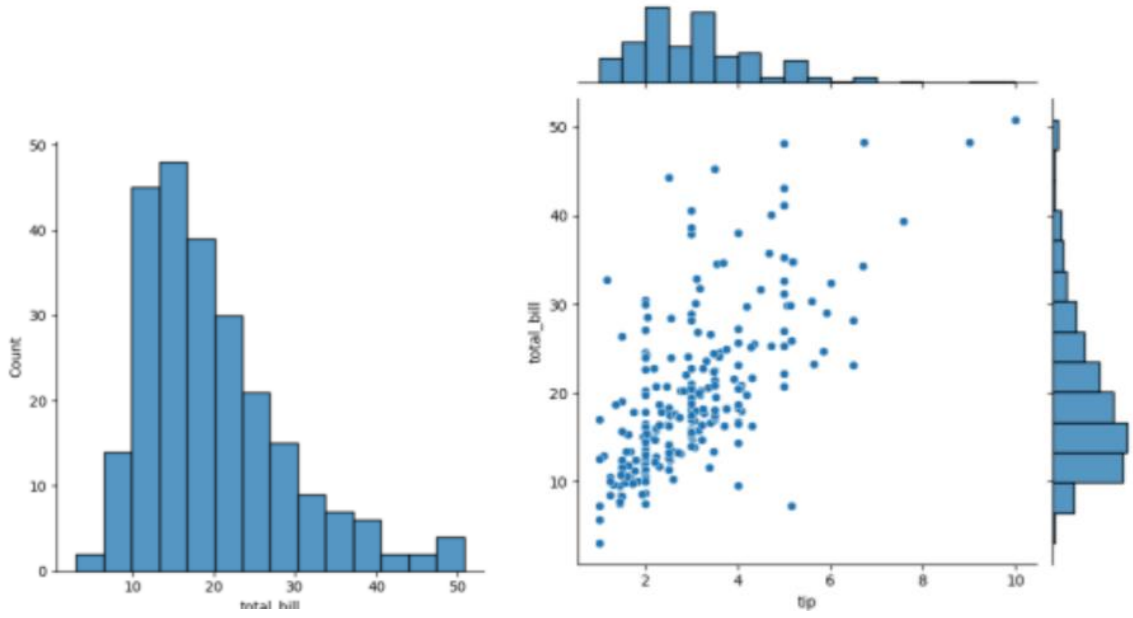
Program:

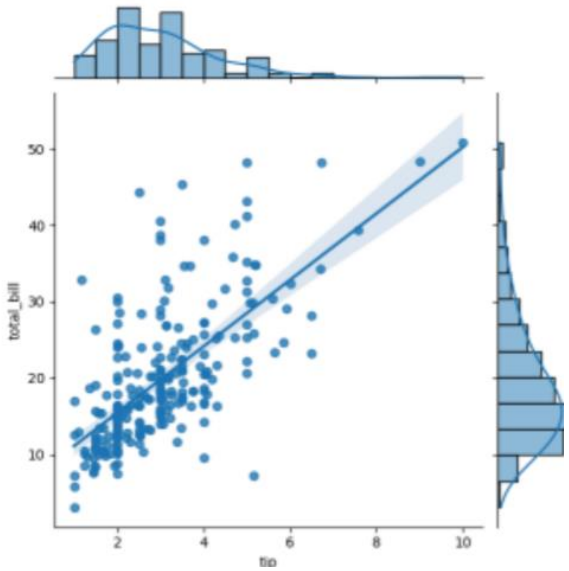
```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
tips = sns.load_dataset('tips')
tips.head()
sns.displot(tips.total_bill, kde=True)
sns.displot(tips.total_bill, kde=False)
sns.jointplot(x=tips.tip, y=tips.total_bill)
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="reg")
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="hex")
sns.pairplot(tips)
tips.time.value_counts()
sns.pairplot(tips, hue='time')
sns.pairplot(tips, hue='day')
sns.heatmap(tips.corr(numeric_only=True), annot=True)
sns.boxplot(tips.total_bill)
sns.boxplot(tips.tip)
sns.countplot(tips.day)
sns.countplot(tips.sex)
tips.sex.value_counts().plot(kind='pie')
tips.sex.value_counts().plot(kind='bar')
sns.countplot(tips[tips.time == 'Dinner']['day'])
```

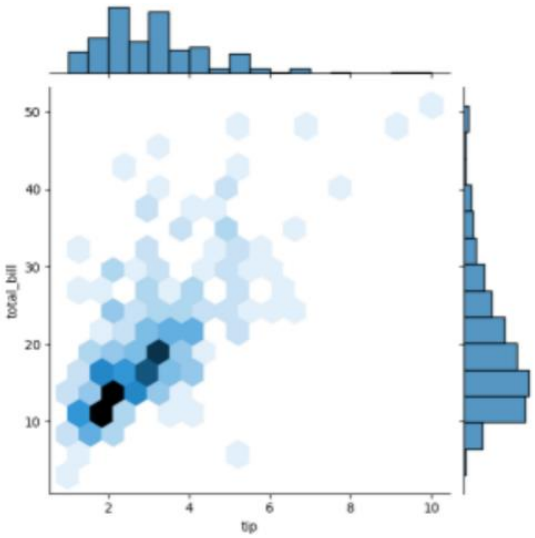
Output:

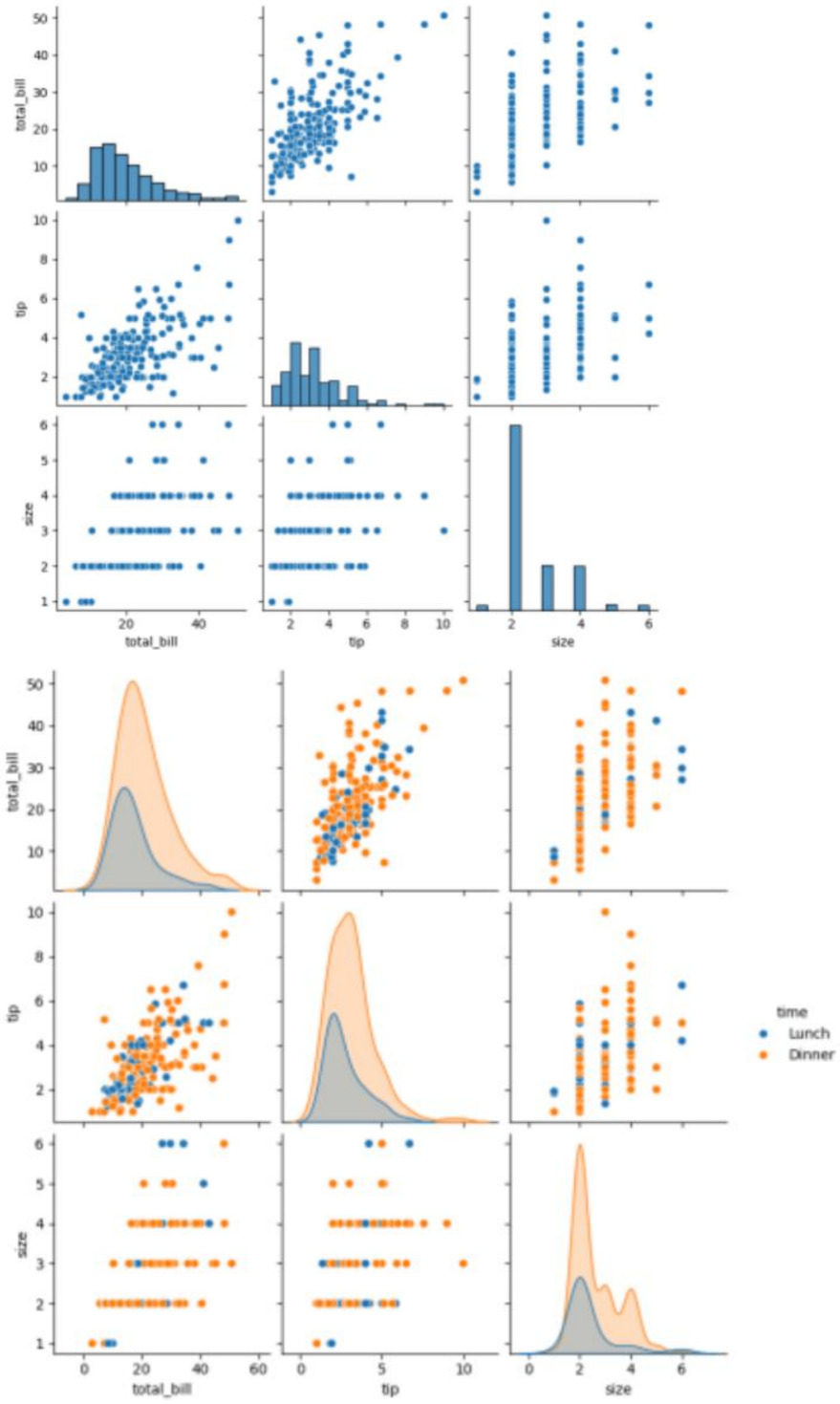
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

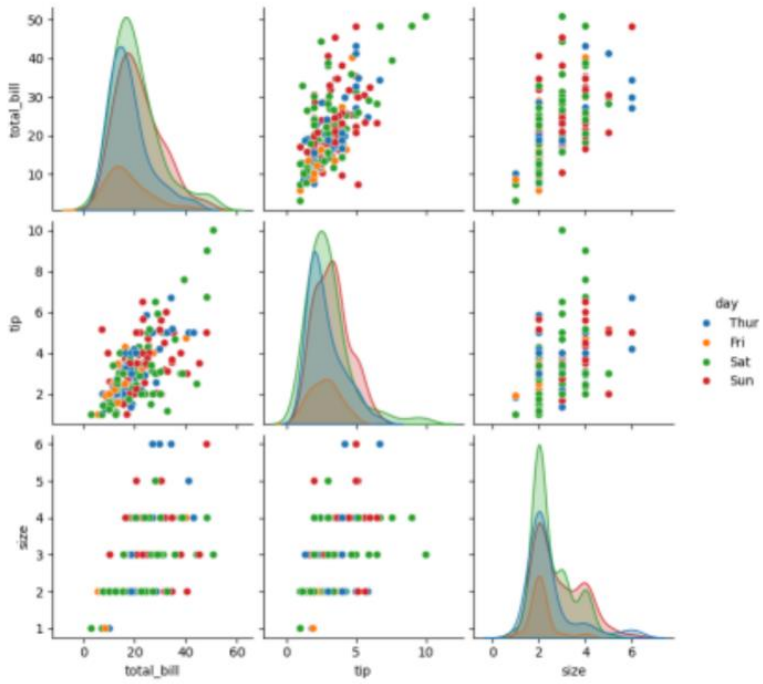




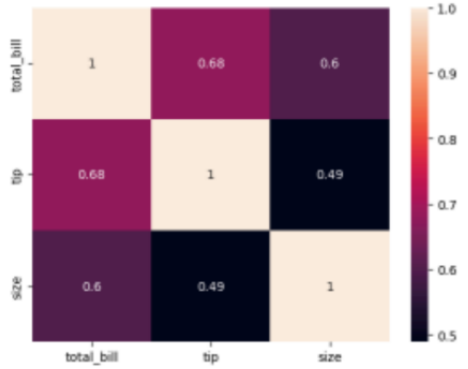


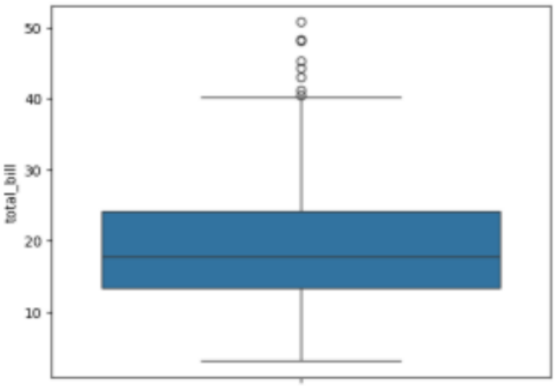


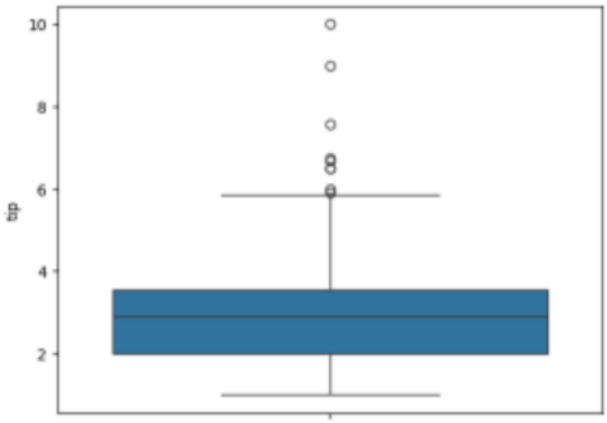

total_bill
tip
size
time
Lunch
Dinner

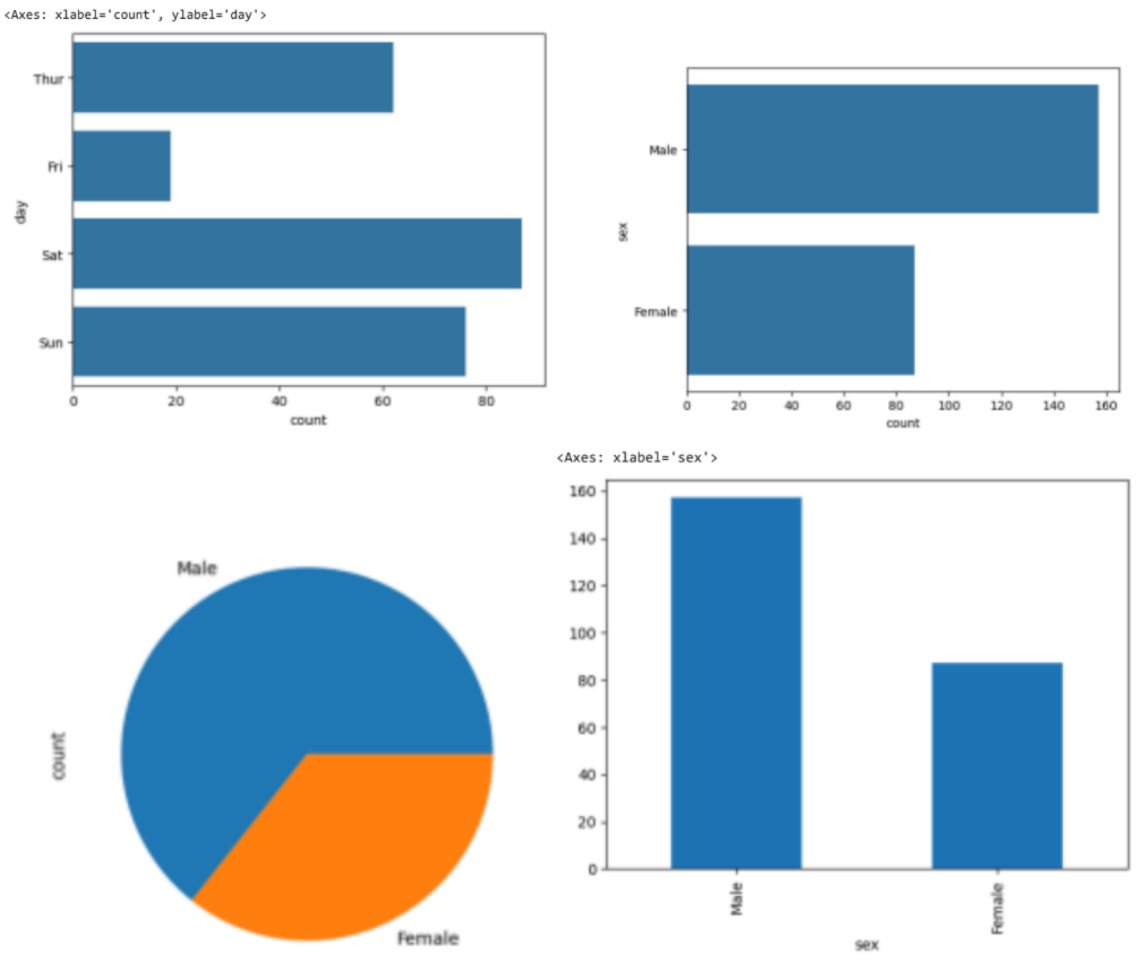

total_bill
tip
size
day
Thur
Fri
Sat
Sun

<Axes: >


total_bill
tip
size
1
0.68
0.6
0.68
1
0.49
0.6
0.49
1


total_bill


tip



Result:

Experiment to understand the EDA is conducted.

8) Experiment to understand Linear regression for the given data set.

Program:

```
import numpy as np
import pandas as pd
df = pd.read_csv('/Salary_data.xlsx - Salary_data.csv')
df
df.info()
df.dropna(inplace=True)
df.info()
df.describe()
features = df.iloc[:, [0]].values
label = df.iloc[:, [1]].values
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=0
)
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(x_train, y_train)
model.score(x_train, y_train)
model.score(x_test, y_test)
model.coef_
model.intercept_
import pickle
pickle.dump(model, open('SalaryPred.model', 'wb'))
model = pickle.load(open('SalaryPred.model', 'rb'))
yr_of_exp = float(input("Enter Years of Experience: "))
yr_of_exp_NP = np.array([[yr_of_exp]])
```

```
Salary = model.predict(yr_of_exp_NP)
print("Estimated Salary for {} years of experience is {}: ".format(yr_of_exp, Salary))
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
Enter Years of Experience: 10.4
Estimated Salary for 10.4 years of experience is [[123630.88046861]]:
```

Result:

Experiment is conducted to understand linear regression of the given data set.

9) Experiment to understand Logistic Regression for a given data set.

Program:

```
import numpy as np
import pandas as pd
df = pd.read_csv('/Social_Network_Ads.xlsx - Social_Network_Ads.csv')
df
df.head()
features = df.iloc[:, [2, 3]].values
label = df.iloc[:, 4].values
features
label
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
for i in range(1, 51):
    x_train, x_test, y_train, y_test = train_test_split(
        features, label, test_size=0.2, random_state=i
    )
    model = LogisticRegression()
    model.fit(x_train, y_train)
    test_score = model.score(x_test, y_test)
    train_score = model.score(x_train, y_train)
    print("Test", test_score, "Train", train_score, "Random State", i)
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=0
)
model = LogisticRegression()
model.fit(x_train, y_train)
model.score(x_train, y_train)
```

```
model.score(x_test, y_test)
from sklearn.metrics import classification_report
y_pred = model.predict(x_test)
print(classification_report(y_test, y_pred))
```

Output:

```
Test 0.8375 Train 0.84375 Random State 1
Test 0.8125 Train 0.853125 Random State 2
Test 0.8375 Train 0.85625 Random State 3
Test 0.9 Train 0.840625 Random State 4
Test 0.8625 Train 0.85 Random State 5
Test 0.8625 Train 0.859375 Random State 6
Test 0.8875 Train 0.8375 Random State 7
Test 0.8125 Train 0.853125 Random State 8
Test 0.8625 Train 0.8375 Random State 9
Test 0.9 Train 0.840625 Random State 10
Test 0.8125 Train 0.84375 Random State 11
Test 0.85 Train 0.853125 Random State 12
Test 0.8375 Train 0.8375 Random State 13
Test 0.8625 Train 0.85625 Random State 14
Test 0.85 Train 0.84375 Random State 15
Test 0.8625 Train 0.85625 Random State 16
Test 0.8375 Train 0.85625 Random State 17
Test 0.875 Train 0.834375 Random State 18
Test 0.85 Train 0.84375 Random State 19
Test 0.875 Train 0.84375 Random State 20
Test 0.8625 Train 0.834375 Random State 21
Test 0.875 Train 0.840625 Random State 22
Test 0.825 Train 0.84375 Random State 23
Test 0.875 Train 0.840625 Random State 24
Test 0.8 Train 0.846875 Random State 25
Test 0.85 Train 0.834375 Random State 26
Test 0.85 Train 0.840625 Random State 27
Test 0.8 Train 0.859375 Random State 28
Test 0.7875 Train 0.853125 Random State 29
Test 0.8625 Train 0.834375 Random State 30
Test 0.8625 Train 0.85625 Random State 31
Test 0.875 Train 0.853125 Random State 32
Test 0.8625 Train 0.84375 Random State 33
Test 0.8 Train 0.8625 Random State 34
Test 0.875 Train 0.83125 Random State 35
Test 0.8625 Train 0.853125 Random State 36
Test 0.8125 Train 0.865625 Random State 37
Test 0.8875 Train 0.840625 Random State 38
Test 0.875 Train 0.8375 Random State 39
Test 0.8375 Train 0.853125 Random State 40
Test 0.775 Train 0.859375 Random State 41
Test 0.8875 Train 0.8375 Random State 42
Test 0.775 Train 0.8625 Random State 43
Test 0.8375 Train 0.85625 Random State 44
Test 0.8375 Train 0.840625 Random State 45
Test 0.875 Train 0.846875 Random State 46
Test 0.9125 Train 0.83125 Random State 47
Test 0.825 Train 0.84375 Random State 48
Test 0.7625 Train 0.8625 Random State 49
```

	precision	recall	f1-score	support
0	0.92	0.97	0.94	58
1	0.89	0.77	0.83	22
accuracy			0.91	80
macro avg	0.91	0.87	0.89	80
weighted avg	0.91	0.91	0.91	80

Result:

Experiment is conducted to understand Logistic Resgression for the given data set.

10) Experiment to understand K-Means Clustering algorithm for a given data set.

Program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

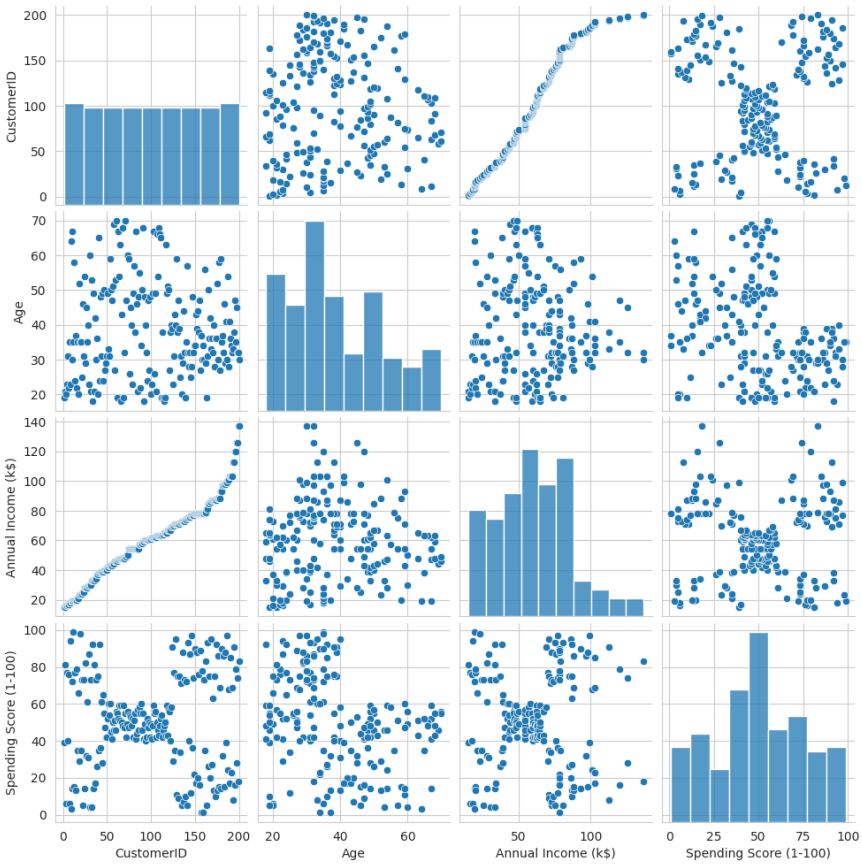

df = pd.read_csv('/Mall_Customers - Mall_Customers.xlsx - Mall_Customers - Mall_Customers.csv')
df.info()
display(df.head())
sns.pairplot(df)
plt.show()
features = df.iloc[:, [3, 4]].values
from sklearn.cluster import KMeans
model = KMeans(n_clusters=5, random_state=0, n_init=10)
model.fit(features)
Final = df.iloc[:, [3, 4]].copy()
Final['label'] = model.predict(features)
display(Final.head())
sns.set_style("whitegrid")
sns.FacetGrid(Final, hue="label", height=8) \
    .map(plt.scatter, "Annual Income (k$)", "Spending Score (1-100)") \
    .add_legend()
plt.show()
features_el = df.iloc[:, [2, 3, 4]].values
wcss = []
for i in range(1, 10):
```

```
    model = KMeans(n_clusters=i, random_state=0, n_init=10)
    model.fit(features_el)
    wcss.append(model.inertia_)
plt.plot(range(1, 10), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
```

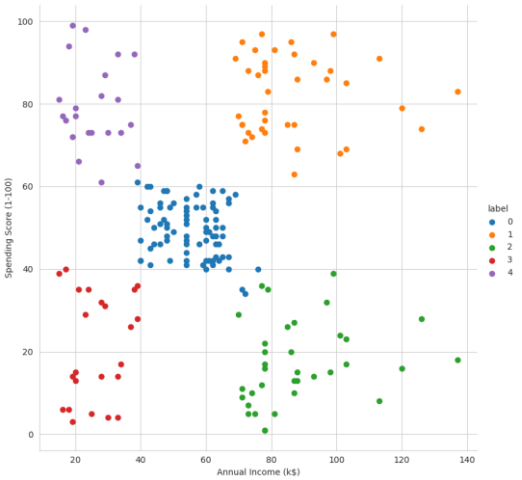
Output:

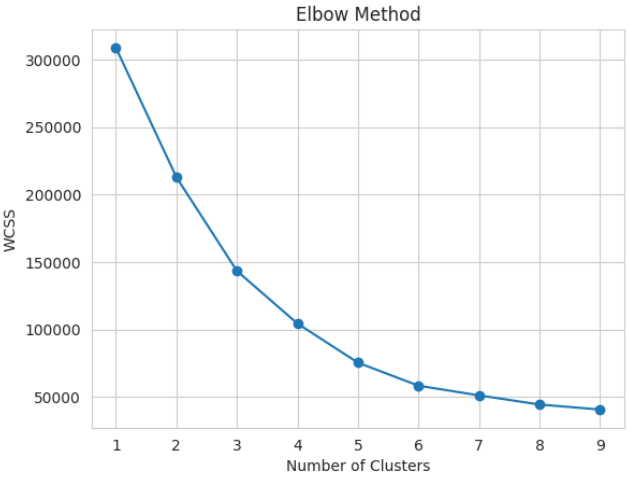
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40



	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	3
1	15	81	4
2	16	6	3
3	16	77	4
4	17	40	3





Result:

Experiment is conducted to understand K-Means Clustering Algorithm of given Data set.

11) Experiment to understand KNN Algorithm for a given data set.

Program:

```
import numpy as np
import pandas as pd
df = pd.read_csv('/Iris (1) - Iris (1).xlsx - Iris (1) - Iris (1).csv')
df.info()
df.variety.value_counts()
df.head()
features = df.iloc[:, :-1].values
label = df.iloc[:, 4].values
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
xtrain, xtest, ytrain, ytest = train_test_split(
    features, label, test_size=0.2, random_state=0
)
model_KNN = KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain, ytrain)
model_KNN.score(xtest, ytest)
model_KNN.score(xtrain, ytrain)
from sklearn.metrics import confusion_matrix, classification_report
ypred = model_KNN.predict(xtest)
print(confusion_matrix(ytest, ypred))
print(classification_report(ytest, ypred))
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
[[11  0  0]
 [ 0 12  1]
 [ 0  0  6]]
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        11
  Versicolor       1.00      0.92      0.96        13
   Virginica       0.86      1.00      0.92         6

    accuracy                           0.97        30
   macro avg       0.95      0.97      0.96        30
weighted avg       0.97      0.97      0.97        30
```

Result:

Experiment is conducted to understand the KNN Algorithm for the given data set.

12) Conduct Z-test for the data given using Python.

Program:

```
import numpy as np
from math import sqrt
from scipy.stats import norm
x_bar = 51.2
mu_0 = 50
sigma = 3
n = 36
z_stat = (x_bar - mu_0) / (sigma / sqrt(n))
p_value = 2 * (1 - norm.cdf(abs(z_stat)))
print(f"Z-statistic: {z_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis → Mean is significantly different from 50 g.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

Output:

```
Z-statistic: 2.400
P-value: 0.0164
Reject Null Hypothesis → Mean is significantly different from 50 g.
```

Result:

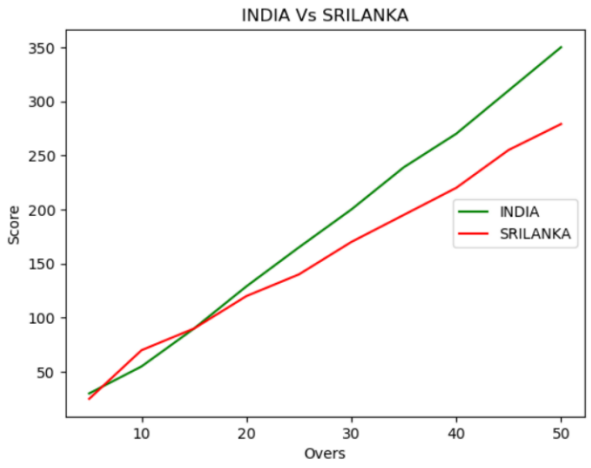
Experiment is conducted for Z-Test using Python.

13.a) Conduct an experiment to show data visualization using line plot.

Program:

```
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```

Output:



Result:

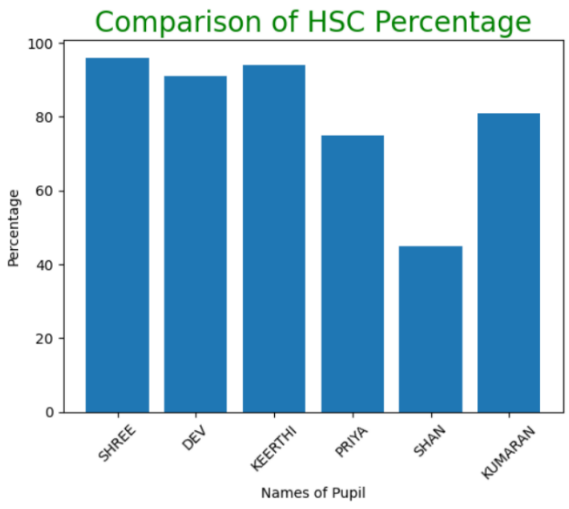
Experiment is conducted to show data Visualization using line plot.

13.b) Conduct an experiment to show data visualization using bar chart.

Program:

```
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```

Output:



Result:

Experiment is conducted to show data visualization using bar chart.

13.c) Conduct an experiment to show data visualization using pie chart.

Program:

```
import matplotlib.pyplot as election

labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']

Votes = [315, 130, 245, 210]

colors = ['green', 'yellow', 'red', 'orange']

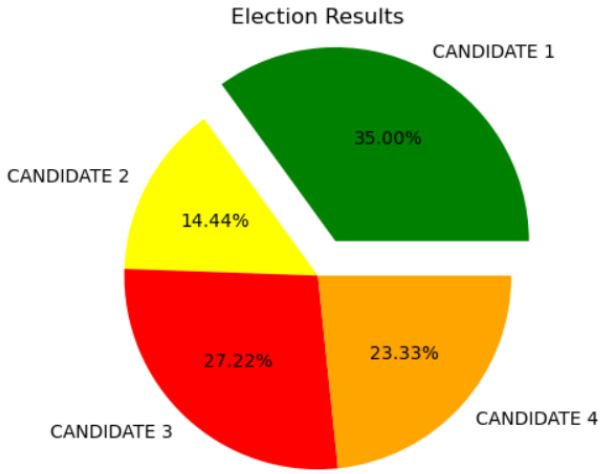
explode = (0.2, 0, 0, 0)

election.pie(Votes, labels=labels, colors=colors, explode=explode,
autopct='%0.2f%%')

election.title('Election Results')

election.show()
```

Output:



Result:

Experiment is conducted to show data visualization using pie chart.

14) To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

Program:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')
sample = gutenberg.raw("austen-emma.txt")
token = word_tokenize(sample)
wlist = []
for i in range(50):
    wlist.append(token[i])
wordfreq = [wlist.count(w) for w in wlist]
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

Output:

```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816',
1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma
', 2), ('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5), ('clever'
, 1), (',', 5), ('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a', 1
), ('comfortable', 1), ('home', 1), ('and', 3), ('happy', 1), ('disposi
tion', 1), (',', 5), ('seemed', 1), ('to', 1), ('unite', 1), ('some', 1
), ('of', 2), ('the', 2), ('best', 1), ('blessings', 1), ('of', 2), ('e
xistence', 1), (';', 1), ('and', 3), ('had', 1), ('lived', 1), ('nearly
', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('the', 2), ('world'
, 1), ('with', 2)]
```

Result:

Thus, the frequency of occurrence of each word in the given text was counted and displayed as word frequency pairs successfully.

15) Data collection and Initial Exploration.

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
np.random.seed(42)
n = 768
df = pd.DataFrame({
    'Pregnancies': np.random.poisson(3, n),
    'Glucose': np.random.normal(120, 30, n).clip(0, 199).astype(int),
    'BloodPressure': np.random.normal(70, 12, n).clip(0, 122).astype(int),
    'SkinThickness': np.random.normal(20, 16, n).clip(0, 99).astype(int),
    'Insulin': np.random.exponential(80, n).clip(0, 846).astype(int),
    'BMI': np.random.normal(32, 7, n).clip(0, 67),
    'DiabetesPedigreeFunction': np.random.gamma(2, 0.25, n),
    'Age': np.random.gamma(5, 6, n).clip(21, 81).astype(int),
    'Outcome': np.random.choice([0, 1], n)
})
print(df.head())
print(df.info())
print(df.describe())
df.hist(figsize=(20, 15))
plt.show()
sns.pairplot(df)
plt.show()
```

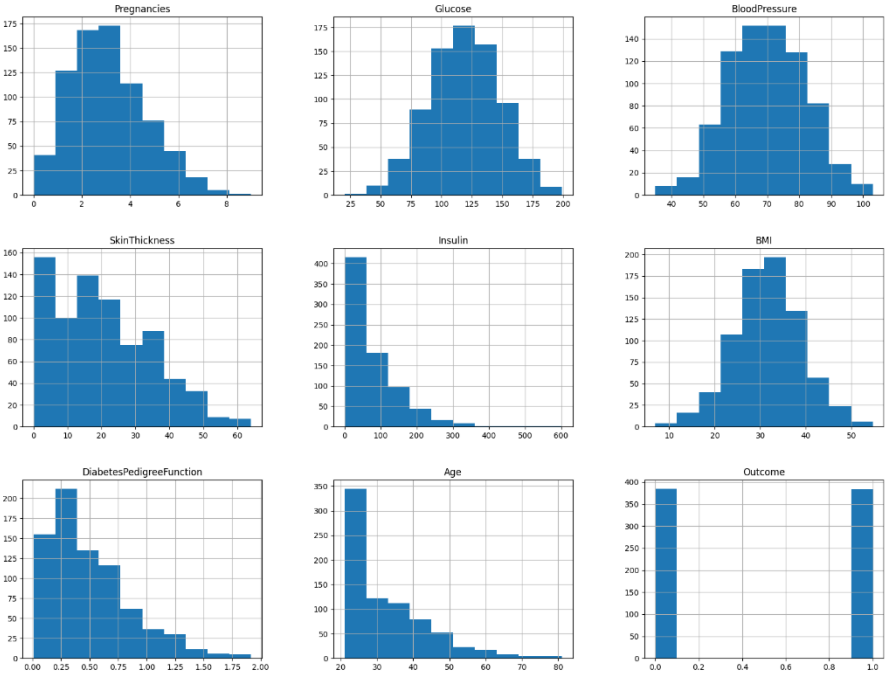
Output:

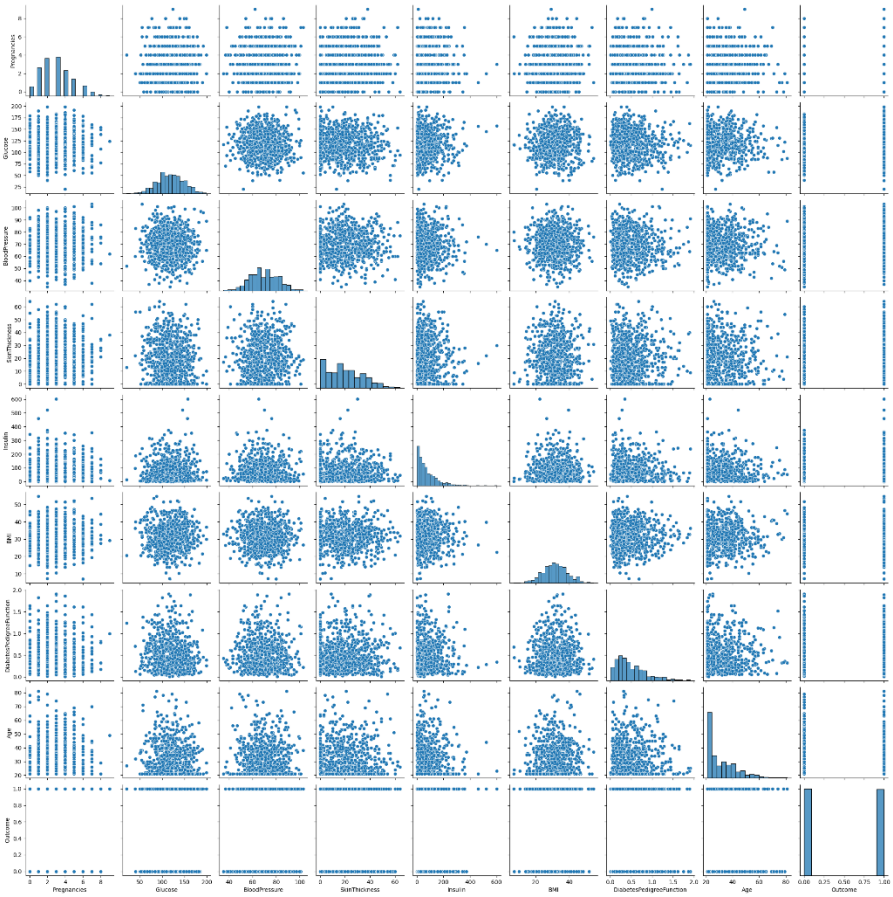
```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin        BMI  \
0            4      165             46             18      353  27.191445
1            1      105             61             46       39  41.748584
2            3      111             78              5      105  29.304742
3            3      104             41              0       92  18.923987
4            2      104             73             20       19  46.095892

   DiabetesPedigreeFunction  Age  Outcome
0                  0.714010   32        0
1                  0.744134   23        0
2                  0.315932   36        1
3                  0.614097   40        0
4                  0.201212   21        0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
```

```
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      2.946615  118.786458      69.653646      20.260417   77.225260
std       1.737241   28.937711      11.942598      14.525162   76.767806
min       0.000000   20.000000      35.000000       0.000000    0.000000
25%       2.000000   99.000000      61.000000       9.000000   22.000000
50%       3.000000  118.000000      70.000000      19.000000   55.000000
75%       4.000000  139.000000      78.000000      30.250000  112.000000
max       9.000000  199.000000     103.000000      64.000000  602.000000

              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000                768.000000  768.000000  768.000000
mean    31.591242                  0.499577   31.697917    0.498698
std      7.400237                  0.357731   11.764747    0.500324
min      6.925562                  0.009226   21.000000    0.000000
25%     26.831742                  0.229507   21.000000    0.000000
50%     31.555204                  0.412649   28.000000    0.000000
75%     36.396015                  0.687550   38.000000    1.000000
max     54.680974                  1.914403   81.000000    1.000000
```


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome



Result:

Thus, data collection and initial exploration were performed successfully.

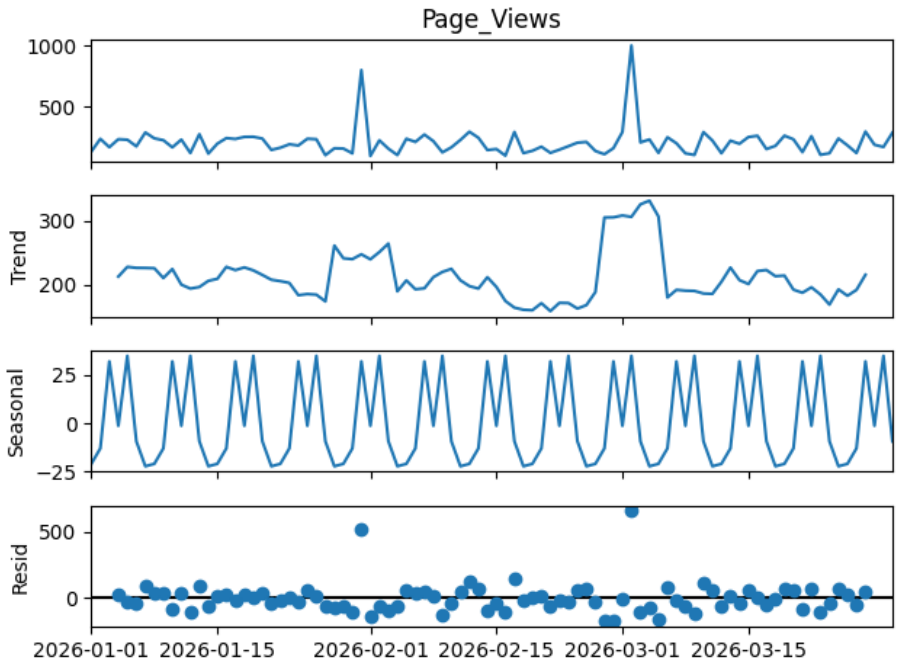
16) Time Series Analysis of Website Traffic.

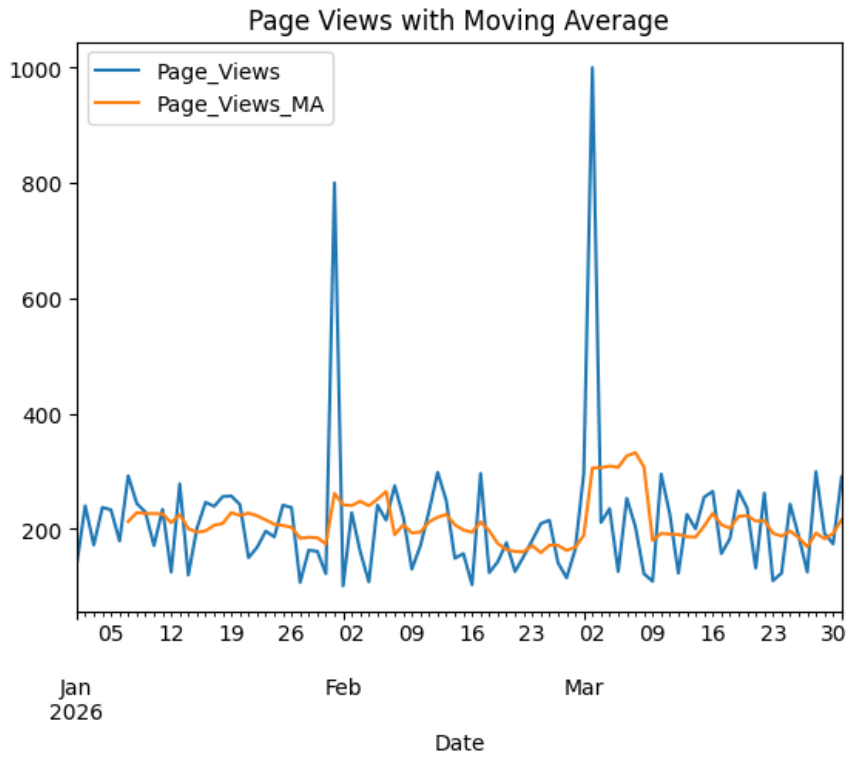
Program:

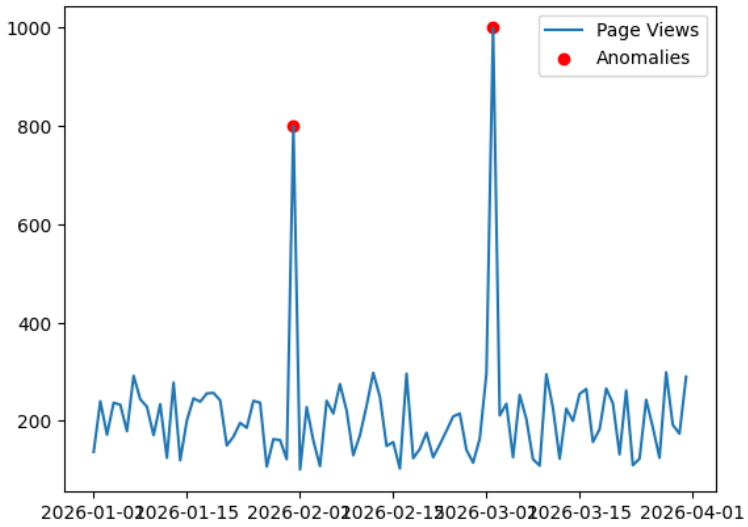
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
np.random.seed(1)
dates = pd.date_range('2026-01-01', periods=90, freq='D')
data = pd.DataFrame({
    'Date': dates,
    'Page_Views': np.random.randint(100, 300, 90),
    'Unique_Visitors': np.random.randint(50, 200, 90),
    'Bounce_Rate': np.random.randint(20, 80, 90)
})
data.set_index('Date', inplace=True)
data.iloc[30, 0] = 800
data.iloc[60, 0] = 1000
decomposition = seasonal_decompose(
    data['Page_Views'], model='additive', period=7
)
decomposition.plot()
plt.show()
data['Page_Views_MA'] = data['Page_Views'].rolling(7).mean()
data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()
```

```
data['Page_Views_Z'] = (
    data['Page_Views'] - data['Page_Views'].mean()
) / data['Page_Views'].std()
anomalies = data[data['Page_Views_Z'].abs() > 3]
plt.plot(data.index, data['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'],
            color='red', label='Anomalies')
plt.legend()
plt.show()
```

Output:







Result:

Thus, Time series analysis of website traffic was performed successfully.

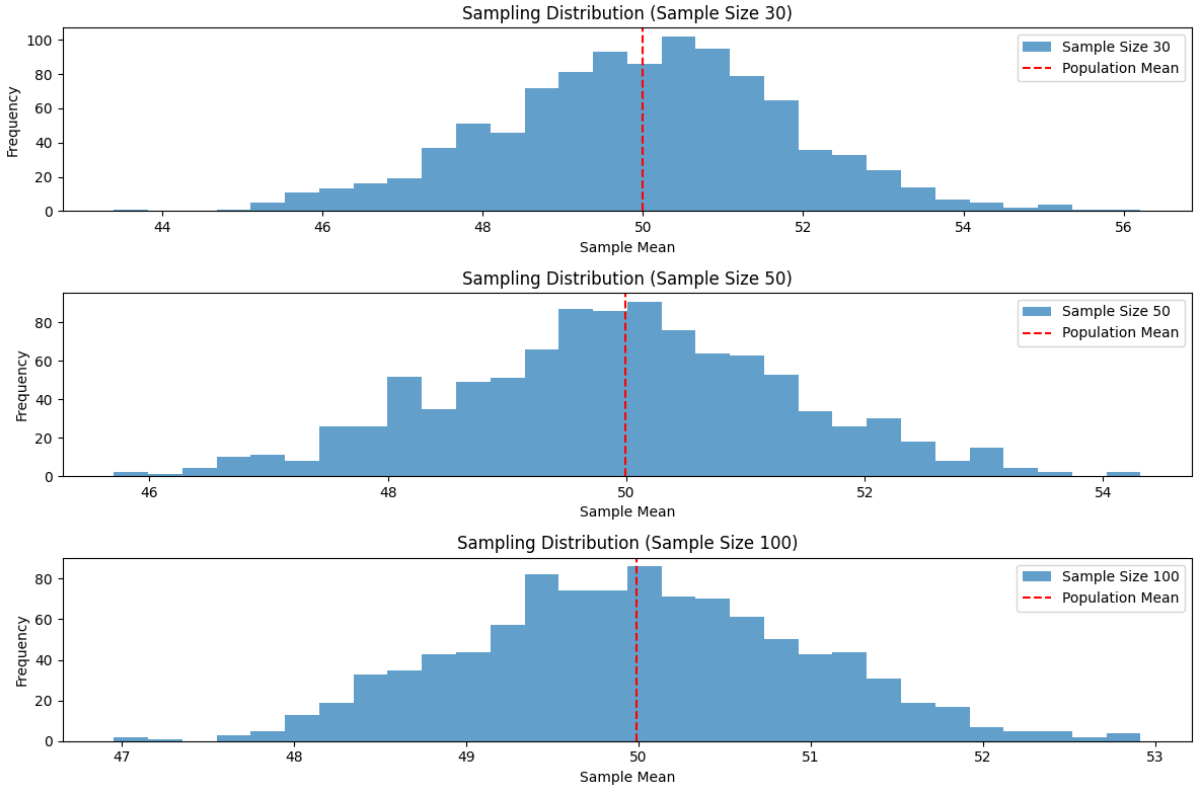
17) Random Sampling and Sampling Distribution.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
population_mean = 50
population_std = 10
population_size = 100000
population = np.random.normal(
    population_mean, population_std, population_size
)
sample_sizes = [30, 50, 100]
num_samples = 1000
sample_means = {}
for size in sample_sizes:
    sample_means[size] = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))
plt.figure(figsize=(12, 8))
for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)
    plt.hist(sample_means[size], bins=30, alpha=0.7,
             label=f'Sample Size {size}')
    plt.axvline(np.mean(population), color='red',
                linestyle='dashed', linewidth=1.5,
                label='Population Mean')
```

```
        plt.title(f'Sampling Distribution (Sample Size {size})')
        plt.xlabel('Sample Mean')
        plt.ylabel('Frequency')
        plt.legend()
    plt.tight_layout()
    plt.show()
```

Output:



Result:

Thus, random sampling and sampling distributions were successfully generated and visualized for different sample sizes.

18) Experiment to understand Matplotlib library use cases in Data Science through visualization.

Program:

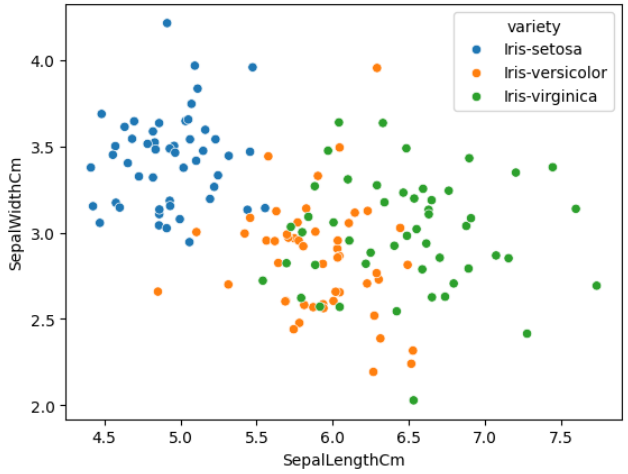
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
np.random.seed(42)
data = pd.DataFrame({
    'SepalLengthCm': np.concatenate([
        np.random.normal(5.0, 0.3, 50),
        np.random.normal(5.9, 0.4, 50),
        np.random.normal(6.5, 0.5, 50)
    ]),
    'SepalWidthCm': np.concatenate([
        np.random.normal(3.4, 0.3, 50),
        np.random.normal(2.8, 0.3, 50),
        np.random.normal(3.0, 0.3, 50)
    ]),
    'PetalLengthCm': np.concatenate([
        np.random.normal(1.5, 0.2, 50),
        np.random.normal(4.3, 0.4, 50),
        np.random.normal(5.5, 0.5, 50)
    ]),
    'PetalWidthCm': np.concatenate([
        np.random.normal(0.2, 0.1, 50),
        np.random.normal(1.3, 0.2, 50),
        np.random.normal(2.0, 0.3, 50)
```

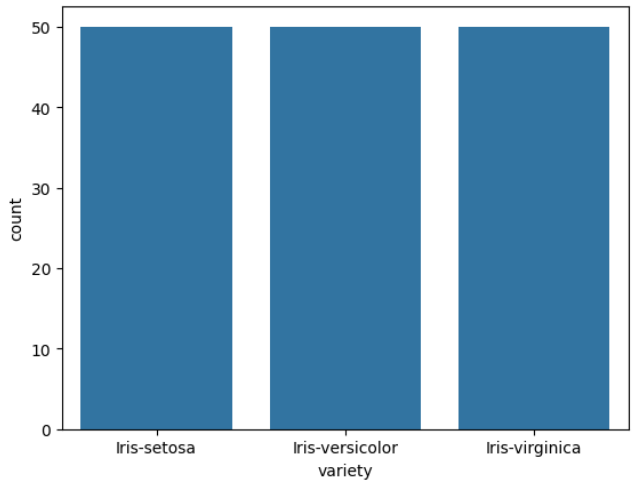
```
    ]),
    'variety': ['Iris-setosa'] * 50 +
               ['Iris-versicolor'] * 50 +
               ['Iris-virginica'] * 50
})
data['Id'] = range(1, 151)
print(data.head())
print(data.info())
print(data.describe())
print(data.value_counts('variety'))
sns.countplot(x='variety', data=data)
plt.show()
sns.scatterplot(x='SepalLengthCm', y='SepalWidthCm',
                hue='variety', data=data)
plt.show()
sns.scatterplot(x='PetalLengthCm', y='PetalWidthCm',
                hue='variety', data=data)
plt.show()
sns.pairplot(data, hue='variety', height=3)
plt.show()
for column in ['PetalLengthCm', 'PetalWidthCm', 'SepalLengthCm']:
    sns.FacetGrid(data, hue='variety', height=5).map(
        sns.histplot, column
    ).add_legend()
    plt.show()
```

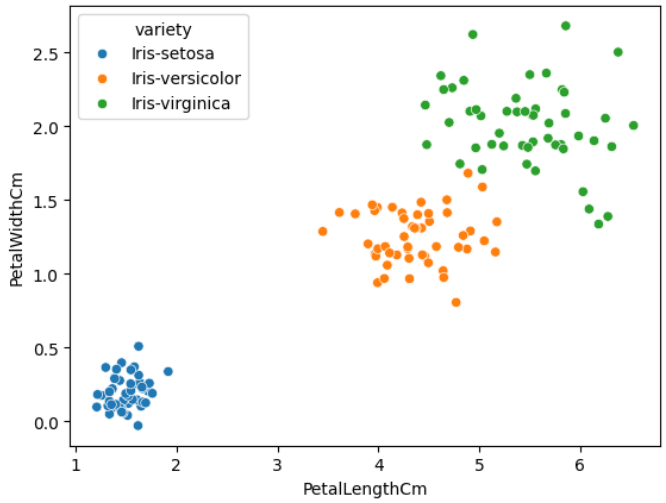
Output:

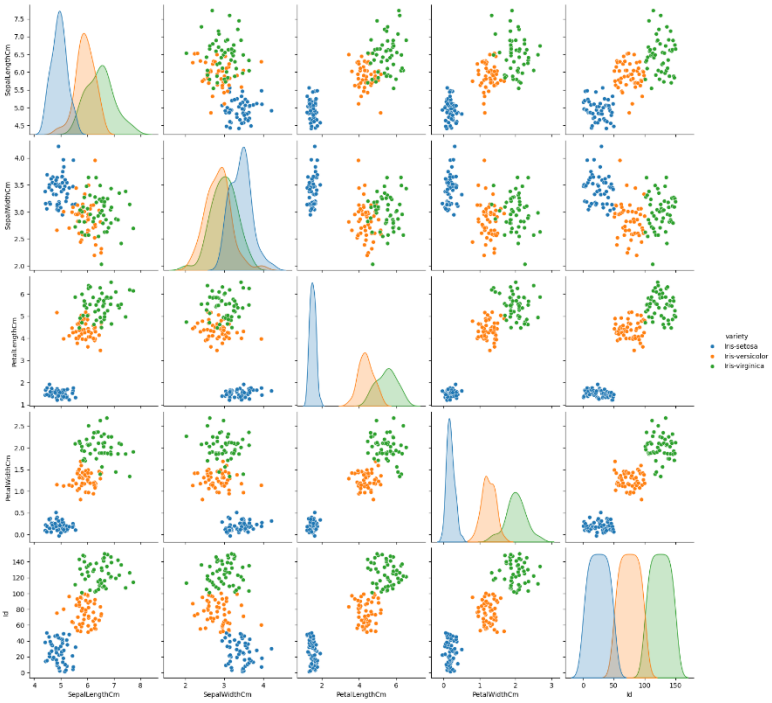
```
   SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      variety  Id
0       5.149014      3.475148       1.334201      0.193732  Iris-setosa   1
1       4.958521      3.503934       1.387964      0.295514  Iris-setosa   2
2       5.194307      3.195993       1.649459      0.101427  Iris-setosa   3
3       5.456909      3.469676       1.622074      0.250405  Iris-setosa   4
4       4.929754      3.487922       1.495820      0.146974  Iris-setosa   5
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   SepalLengthCm  150 non-null    float64
 1   SepalWidthCm   150 non-null    float64
 2   PetalLengthCm  150 non-null    float64
 3   PetalWidthCm   150 non-null    float64
 4   variety        150 non-null    object
 5   Id             150 non-null    int64
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
None
       SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm          Id
count     150.000000    150.000000     150.000000    150.000000  150.000000
mean        5.773275      3.088036       3.784888      1.150117   75.500000
std         0.749582      0.392937       1.723460      0.772978   43.445368
min         4.412099      2.027620       1.210383     -0.030192    1.000000
25%         5.096477      2.820749       1.625035      0.258596   38.250000
50%         5.809148      3.081515       4.349614      1.237650   75.500000
75%         6.298614      3.379593       5.106935      1.865582  112.750000
max         7.731621      4.216051       6.530374      2.681208  150.000000
```

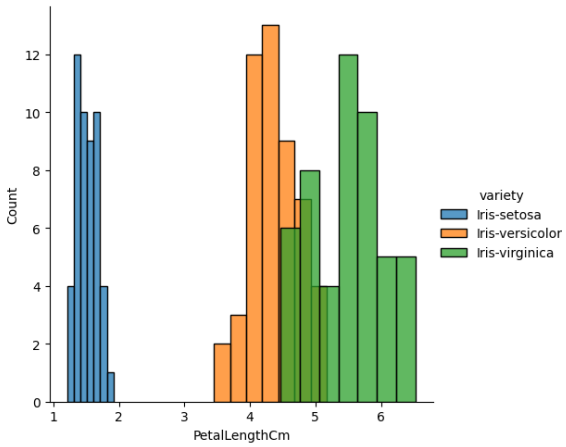
```
variety
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

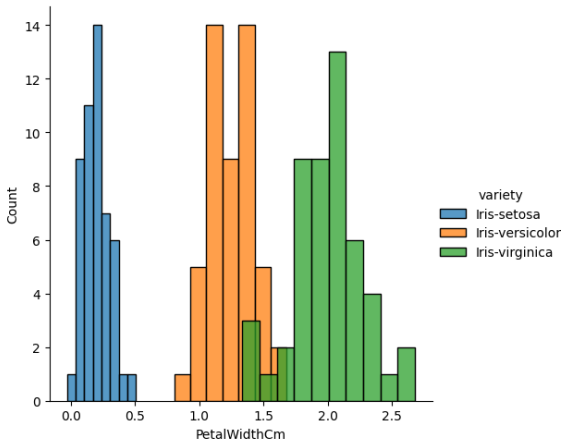


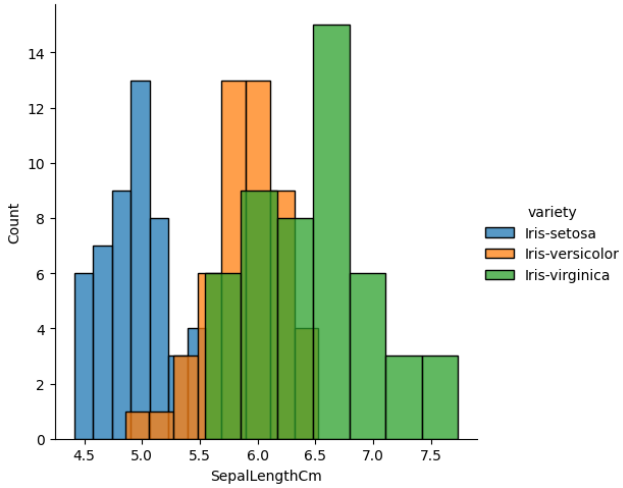



variety
Iris-setosa
Iris-versicolor
Iris-virginica
PetalWidthCm
PetalLengthCm


SepalLengthCm
SepalWidthCm
PetalLengthCm
PetalWidthCm
Id
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
PetalLengthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
PetalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica



Result:

Thus, the Matplotlib and Seaborn libraries were used successfully to visualize the dataset using count plots, scatter plots, pairplots, and histograms.

19) Experiment to understand array function in Data science.

Program:

```
import numpy as np
array = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90])
print(array)
print(np.sqrt(array))
print(array.ndim)
new_array = array.reshape(3, 3)
print(new_array)
print(new_array.ndim)
print(new_array.ravel())
newm = new_array.reshape(3, 3)
print(newm[2, 1:3])
print(newm[1:2, 1:3])
print(new_array[0:3, 0:1])
print(new_array[1:3])
```

Output:

```
[10 20 30 40 50 60 70 80 90]
[3.16227766 4.47213595 5.47722558 6.32455532 7.07106781 7.74596669
 8.36660027 8.94427191 9.48683298]
1
[[10 20 30]
 [40 50 60]
 [70 80 90]]
2
[10 20 30 40 50 60 70 80 90]
[80 90]
[[50 60]]
[[10]
 [40]
 [70]]
[[40 50 60]
 [70 80 90]]
```

Result:

Thus, NumPy array functions such as square root, dimensions, reshaping, flattening, and slicing were performed successfully.

20) Experiment to understand pandas library use cases in Data science.

Program:

```
import numpy as np
import pandas as pd
data = [ [1, 'Smith', 5000],
    [2, 'Jones', 6000],
    [3, 'David', 7000],
    [4, 'Priya', 5000],
    [5, 'Ravi', 8000],
    [6, 'Anu', 3000],
    [7, 'Kumar', 6000]]
df = pd.DataFrame(data)
df.columns = ['Emp_id', 'Name', 'Salary']
print(df)
print(df.head())
print(df.tail())
df.info()
print(df['Salary'])
print("Mean:", df['Salary'].mean())
print("Median:", df['Salary'].median())
print("Mode:")
print(df['Salary'].mode())
print("Variance:", df['Salary'].var())
print("Standard Deviation:", df['Salary'].std())
print(df.describe())
print(df.describe(include='all'))
print(df.columns)
```

print(df.values)

Output:

```
   Emp_id   Name  Salary
0       1  Smith    5000
1       2  Jones    6000
2       3  David    7000
3       4  Priya    5000
4       5   Ravi    8000
5       6    Anu    3000
6       7  Kumar    6000
   Emp_id   Name  Salary
0       1  Smith    5000
1       2  Jones    6000
2       3  David    7000
3       4  Priya    5000
4       5   Ravi    8000
   Emp_id   Name  Salary
2       3  David    7000
3       4  Priya    5000
4       5   Ravi    8000
5       6    Anu    3000
6       7  Kumar    6000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Emp_id  7 non-null      int64
 1   Name    7 non-null      object
 2   Salary  7 non-null      int64
dtypes: int64(2), object(1)
memory usage: 300.0+ bytes
0    5000
1    6000
2    7000
3    5000
4    8000
5    3000
6    6000
Name: Salary, dtype: int64
Mean: 5714.285714285715
Median: 6000.0
Mode:
0    5000
1    6000
Name: Salary, dtype: int64
Variance: 2571428.5714285714
Standard Deviation: 1603.5674514745463
```

	Emp_id	Salary
count	7.000000	7.000000
mean	4.000000	5714.285714
std	2.160247	1603.567451
min	1.000000	3000.000000
25%	2.500000	5000.000000
50%	4.000000	6000.000000
75%	5.500000	6500.000000
max	7.000000	8000.000000

	Emp_id	Name	Salary
count	7.000000	7	7.000000
unique	NaN	7	NaN
top	NaN	Smith	NaN
freq	NaN	1	NaN
mean	4.000000	NaN	5714.285714
std	2.160247	NaN	1603.567451
min	1.000000	NaN	3000.000000
25%	2.500000	NaN	5000.000000
50%	4.000000	NaN	6000.000000
75%	5.500000	NaN	6500.000000
max	7.000000	NaN	8000.000000

```
Index(['Emp_id', 'Name', 'Salary'], dtype='object')
[[1 'Smith' 5000]
 [2 'Jones' 6000]
 [3 'David' 7000]
 [4 'Priya' 5000]
 [5 'Ravi' 8000]
 [6 'Anu' 3000]
 [7 'Kumar' 6000]]
```

Result:

Thus, the Pandas library was used successfully to create and explore a DataFrame and perform statistical operations on given dataset.